Language modeling, perplexity and tokenization

Large Language Models · Module 1, lecture 1 (M1P1) · Week 1

Heudson Mirandola

Instituto de Matemática, UFRJ

October 2026

One idea organizes the lecture

Modeling language, predicting the next symbol
and compressing text are the same problem.

- **Section 2:** the loss we train with *is* the number of bits a compressor built on the model would spend.
- **Section 3:** the first concrete model (n-grams) and the wall it hits.
- **Section 4:** the tokenizer is itself a compressor — and leaves visible marks on production models.

Objectives

By the end of the lecture you should be able to

1. write the likelihood of an autoregressive model and say why training is minimizing cross-entropy;
2. compute and interpret perplexity — and know when it is *not* comparable;
3. explain, via arithmetic coding, why modeling language is compressing;
4. estimate a smoothed n-gram model and name its two structural limits;
5. run BPE by hand and predict side effects of tokenization.

Prerequisites: discrete probability; entropy, cross-entropy, KL; maximum likelihood.
Conventions: log is natural, $\log_2$ when the unit is the bit; $x_{<t} = (x_1, \ldots, x_{t-1})$, $x_{1:T} = (x_1, \ldots, x_T)$.

Outline

1. The problem
2. How to measure
3. *n*-grams
4. Tokenization
5. Bridge
6. Appendix: proofs and code

A language model is a distribution over sequences

Fix a finite **vocabulary** $\mathcal{V}$, $|\mathcal{V}| = V$. A text is a finite sequence $x = (x_1, \ldots, x_T)$, $x_t \in \mathcal{V}$. All such sequences: $\mathcal{V}^*$.

$$p : \mathcal{V}^* \to [0, 1], \qquad \sum_{x \in \mathcal{V}^*} p(x) = 1.$$

- Tokens are discrete: "density" in the syllabus means probability mass function.
- What a symbol *is* (letter, byte, word, subword) is a design decision — Section 4.
- $\mathcal{V}^*$ is infinite: p does not fit in a table. We need a computable form.

Autoregressive factorization

Chain rule — valid for *any* joint distribution, no assumption:

$$p(x_1, \ldots, x_T) \;=\; \prod_{t=1}^{T} p(x_t \mid x_{<t}).$$

Each factor is a V-**way classification** conditioned on the prefix.
An autoregressive LM is a classifier: prefix in, V probabilities out.

The whole course up to Module 6 is about how to parametrize and estimate $p_\theta(x_t \mid x_{<t})$: count tables (Section 3) or a neural network (Week 2).

Detail 1: the end of the sequence is a symbol

$\mathcal{V} = \{a, b\}$. What is $p(a)$, the probability of the length-1 sequence?

By the factorization, $p(a) = p(x_1 = a)$. But

$$p(a, a) + p(a, b) = p(x_1 = a)\big[p(x_2 = a \mid a) + p(x_2 = b \mid a)\big] = p(x_1 = a) = p(a).$$

The mass of (a) equals the mass of *all* length-2 sequences starting with a. Summing over lengths, the total diverges.

p is not a distribution over $\mathcal{V}^*$ — it is one distribution per length.

Detail 1: the fix

Add a symbol <eos> and define: the sequence *ends* when <eos> is emitted.

$$p(a) = p(x_1 = a)\, p(x_2 = \texttt{<eos>} \mid a) \;\neq\; p(x_1 = a).$$

- Now p sums to 1 over $\mathcal{V}^*$ (under mild conditions on <eos>).
- Every practical model has this symbol.
- Without it, the model "does not know how to stop".

Detail 2: generation is ancestral sampling

$$x_1 \sim p_\theta(\cdot \mid \varnothing), \qquad x_2 \sim p_\theta(\cdot \mid x_1), \qquad x_3 \sim p_\theta(\cdot \mid x_1, x_2), \qquad \ldots \text{ until } \texttt{<eos>}.$$

- Temperature, top-k, top-p, beam search (Module 7) modify this sampling — and each one changes the distribution actually sampled.
- Generating is *one* use of p_θ. Scoring a text, ranking continuations, compressing are others.

Detail 3: the factorization is not unique

The chain rule holds in any order of the positions.

- Left→right is convenience: text is produced that way.
  It defines the **decoder-only** family (GPT and descendants).
- **Encoder-only** (BERT, Week 3) abandons the factorization and models $p(x_t \mid x_{\neq t})$: one symbol given all the others.
  Not a joint distribution in any simple way — but useful for representing text.

The target: approximate $p^\star$

"A good model gives high probability to plausible text." *Plausible according to whom?*

Working hypothesis of the field. There is a distribution $p^\star$ over $\mathcal{V}^*$ — the process that generated the corpus — and the corpus is a sample

$$x^{(1)}, \ldots, x^{(N)} \sim p^\star.$$

Goal: choose θ so that $p_\theta \approx p^\star$.

Idealized (there is no "the English language"), but useful: it makes "approximate" precise, and its consequences hold in practice. Next section.

Outline

1. The problem
2. How to measure
3. *n*-grams
4. Tokenization
5. Bridge
6. Appendix: proofs and code

Maximum likelihood is cross-entropy

Concatenate the corpus into one sequence $x_{1:T}$ (<eos> between documents).

Log-likelihood, sign flipped, divided by T: the MLE minimizes

$$\mathcal{L}(\theta) \;=\; -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}),$$

the **average cross-entropy per token** — nats with ln, bits with $\log_2$.

This is literally `F.cross_entropy` in Lab 3: V logits, softmax, $-\log$ of the probability of the correct token.

Why "cross-entropy"

For distributions q, r on a finite set:

$$H(q, r) := -\sum_x q(x) \log r(x) \;=\; H(q) + \mathrm{KL}(q \,\|\, r).$$

Check: $-\sum q \log r = -\sum q \log q + \sum q \log(q/r)$.

Apply at each position, with $p_t^\star := p^\star(\cdot \mid x_{<t})$ and $p_\theta^t := p_\theta(\cdot \mid x_{<t})$:

$$\mathbb{E}_{x_t \sim p_t^\star}\big[-\log p_\theta(x_t \mid x_{<t})\big] \;=\; \underbrace{H(p_t^\star)}_{\text{entropy of the source}} + \underbrace{\mathrm{KL}(p_t^\star \,\|\, p_\theta^t)}_{\text{model error}}.$$

What the decomposition says

- $H(p_t^\star)$ does not depend on θ: intrinsic uncertainty of the next symbol.
- $\mathrm{KL} \geq 0$, zero iff $p_\theta^t = p_t^\star$.
- So minimizing cross-entropy = minimizing KL to $p^\star$.
- The lowest attainable loss is H — **not zero**. After "the cat slept on the", both "couch" and "rug" are fine.
- "How much is left to learn" is $\mathcal{L} - H$, and H is unknown.

Hook for Module 3

Chinchilla: $L(N, D) = E + A/N^\alpha + B/D^\beta$. The constant E *is* the entropy of the source.

Numerical example

Five tokens, probabilities 0.5, 0.1, 0.25, 0.05, 0.8 (each given the correct prefix).

$$\begin{aligned}\mathcal{L} &= -\tfrac{1}{5}\big(\ln 0.5 + \ln 0.1 + \ln 0.25 + \ln 0.05 + \ln 0.8\big)\\ &= -\tfrac{1}{5}\big(-0.693 - 2.303 - 1.386 - 2.996 - 0.223\big)\\ &\approx 1.520 \text{ nats } \approx \text{ 2.19 bits per token.}\end{aligned}$$

- The 0.05 token contributes 2.996: almost 40% of the sum.
- **One very unlikely token outweighs several well-predicted ones.**
- In the limit: a zero-probability token makes the loss infinite.

Entropy rate

For a *stationary* process (X_t):

$$H(\mathcal{X}) \;=\; \lim_{T\to\infty} \frac{1}{T} H(X_1, \ldots, X_T) \;=\; \lim_{T\to\infty} H(X_T \mid X_{<T}).$$

Uncertainty *per symbol* when the context grows without bound.

Shannon–McMillan–Breiman. Stationary and ergodic $\Rightarrow$

$$-\frac{1}{T} \log p^\star(X_{1:T}) \xrightarrow[T\to\infty]{\text{a.s.}} H(\mathcal{X}).$$

A time average converges to an ensemble average. With p_θ: limit $= H +$ KL rate.

What SMB buys us

One long test corpus is enough to evaluate a language model.
The average along the sequence estimates the population quantity.

Two honest caveats:

- Natural language is not stationary (position, topic, period).
- Even so, this is what everyone reports, and comparisons are informative.
  The theory guides; it does not apply literally.

Proof: Cover & Thomas, 16.8. The i.i.d. case is in Appendix A.2.

Shannon's experiment (1951)

Ask a person to guess the *next letter* of a text; count the guesses until correct. The distribution of guesses bounds the entropy.

- 27 symbols (26 letters + space): uniform would be $\log_2 27 \approx 4.75$ bits/letter.
- Result: about **1 bit per letter** (between 0.6 and 1.3).
- English is highly redundant. That redundancy is what an LM captures.

Current neural models: below 1 bit/character on `enwik8`. Seventy years later, consistent.

Perplexity

$$\mathrm{PPL} \;=\; \exp(\mathcal{L}) \;=\; \Big(\prod_{t=1}^{T} p_\theta(x_t \mid x_{<t})\Big)^{-1/T}$$

geometric mean of the inverse probabilities of the correct tokens.

- **Effective number of alternatives**: $\mathrm{PPL} = 20$ means "hesitating as if choosing uniformly among 20".
- Uniform over V: $\mathrm{PPL} = V$. Always right: $\mathrm{PPL} = 1$. Our example: $e^{1.520} \approx 4.57$.
- A monotone transform of $\mathcal{L}$ — no new information, more interpretable, not additive (Exercise 1).

Pitfall 1: per-token perplexity depends on the tokenizer

Same text. Model A: 32k vocabulary. Model B: 128k.

- B's tokenizer uses *fewer* tokens; each token covers more characters.
- Predicting one of B's tokens is harder: more information per step.
- **B's per-token perplexity tends to be higher even if B compresses better.**

Normalize by the amount of *text*, not of tokens:

$$\text{bits per byte} = \frac{\mathcal{L}_{\text{nats/token}}}{\ln 2} \cdot \frac{\#\text{tokens}}{\#\text{bytes}}.$$

Pitfall 1: the numbers

A 4200-byte text.

	tokens	$\mathcal{L}$ (nats/token)	PPL	bits/byte
Model 1	1000	3.0	20	$\frac{3.0}{0.693} \cdot \frac{1000}{4200} \approx 1.03$
Model 2	800	3.5	33	$\frac{3.5}{0.693} \cdot \frac{800}{4200} \approx 0.96$

Model 2: higher perplexity, **better compression**.

- Serious pretraining reports give bits per byte.
- **Lab 1 and Lab 3 report both.** Otherwise *n*-gram vs. GPT is meaningless.

Pitfall 2: how much context the model sees

Window of 1024 tokens, text of 10k tokens.

- **Disjoint blocks.** Cut into blocks of 1024. The first token of each block sees nothing; on average, 512 tokens of context.
- **Sliding window.** Each token sees the previous 1023. One pass per token: expensive, but maximal context.

The second gives lower perplexity. Neither is wrong; comparing them is.
Document which one was used.

Pitfall 3: a single zero destroys everything

$$p_\theta(x_t \mid x_{<t}) = 0 \text{ for one } t \quad \Longrightarrow \quad \mathcal{L} = \infty,\ \mathrm{PPL} = \infty.$$

Two consequences that reappear later:

- count-based models need **smoothing** (Section 3);
- **byte-level** tokenization (Section 4) makes every text representable.

In code

```
# log_probs: log p(x_t | x_<t) in nats, one per token.
# n_bytes:   len(text.encode("utf-8"))
import math

def cross_entropy(log_probs):              # nats per token
    return -sum(log_probs) / len(log_probs)

def perplexity(log_probs):                 # "effective number of alternatives"
    return math.exp(cross_entropy(log_probs))

def bits_per_byte(log_probs, n_bytes):     # independent of the tokenizer
    total_bits = -sum(log_probs) / math.log(2)
    return total_bits / n_bytes
```

Modeling language is compressing

Claim

Cross-entropy is not just *a* reasonable metric.
It is the length, in bits, of the code a compressor built on the model would produce.

1. A language model is a compressor — via arithmetic coding.
2. A compressor is a language model — via Kraft.

Then: evidence, and why it matters for the course.

Prefix codes and Kraft

A binary code gives symbol x a codeword of $\ell(x)$ bits. **Prefix-free**: no codeword is a prefix of another $\Rightarrow$ decodable without separators.

Kraft–McMillan. A prefix code with lengths $\ell(x)$ exists iff

$$\sum_x 2^{-\ell(x)} \;\le\; 1.$$

For a distribution q: expected length $\ge H_2(q)$, and the *Shannon code* $\ell(x) = \lceil -\log_2 q(x) \rceil$ is within 1 bit of it.

$-\log_2 q(x)$ is the number of bits a good code spends on x.

From symbols to sequences: arithmetic coding

Symbol by symbol wastes up to 1 bit per symbol (rounding). One codeword per sequence is impossible.

Arithmetic coding: the whole sequence becomes *one* number in $[0, 1)$, using

$$< -\log_2 p(x_{1:T}) + 2 \text{ bits in total,}$$

and it consumes $p(\cdot \mid x_{<t})$ **one step at a time, in order** — exactly what an autoregressive model provides.

Mechanism: keep an interval, initially $[0, 1)$. At step t, split it into V slices proportional to $p(\cdot \mid x_{<t})$ and keep the slice of x_t.

Arithmetic coding: example

$\mathcal{V} = \{a, b, \text{<eos>}\}$, fixed model $p(a) = 0.5$, $p(b) = 0.3$, $p(\text{<eos>}) = 0.2$. Encode $a\,b\,$<eos>:

Step	Symbol	Interval	
0	—	$[0,\ 1)$	
1	a	$[0,\ 0.5)$	first half
2	b	$[0.25,\ 0.40)$	slice of b inside $[0,\ 0.5)$
3	<eos>	$[0.37,\ 0.40)$	slice of <eos> inside $[0.25,\ 0.40)$

- Final width $0.03 = 0.5 \cdot 0.3 \cdot 0.2 = p(a\,b\,\text{<eos>})$. Always.
- $-\log_2 0.03 \approx 5.06$; with 6 bits, $0.375 = 0.011000_2$ falls inside.
- The decoder redoes the splits — it needs the same model.

Direction 1: an LM is a compressor

With context, nothing changes: the slices at step t are $p_\theta(\cdot \mid x_{<t})$, computed from the prefix, as in generation.

$$\frac{\ell(x_{1:T})}{T} \approx \mathcal{L}_{\text{bits/token}}.$$

Cross-entropy in bits **is** the compression rate.
"Train an LM" and "build an adaptive lossless compressor" are the same problem.

The < 2 bits of waste do not grow with T. *The decoder needs θ: model size enters the accounting — see below.*

Direction 2: a compressor is an LM

Any lossless compressor C (say `gzip`) defines

$$p_C(x) \propto 2^{-|C(x)|}.$$

- By Kraft, $\sum_x 2^{-|C(x)|} \leq 1$: a sub-distribution. Normalizing is hard — unnecessary for ranking.
- Given a prefix, the symbol w minimizing $|C(x_{<t}\, w)|$ is p_C's favorite.
- `gzip` predicts the next character non-trivially: a legitimate, cheap baseline (Exercise 5).

Evidence: Delétang et al. (ICLR 2024)

LMs of several sizes + arithmetic coder, on text, images (byte patches), audio.

- A 70B model trained *on text only* compresses text to ∼8% (`gzip`: ∼32%) — and beats PNG on images and FLAC on audio.
- With the model size added to the code (*adjusted* rate), the 70B model only wins when the corpus is much larger than the model.

Chinchilla again: the optimal model size depends on the amount of data.

Why the compression lens matters

1. A non-anthropomorphic reading of "the model understands": **understanding** = **predicting** = **compressing**.
2. It explains why one scalar — the pretraining loss — is such a good measure of general capability.
3. The tokenizer is a **first-stage compressor**; the model compresses what is left. Section 4.

Outline

1. The problem
2. How to measure
3. *n*-grams
4. Tokenization
5. Bridge
6. Appendix: proofs and code

The Markov assumption

Truncate the context to the $n-1$ previous symbols:

$$p(x_t \mid x_{<t}) \;\approx\; p(x_t \mid x_{t-n+1}, \ldots, x_{t-1}).$$

- $n = 1$ unigram (independent symbols); $n = 2$ bigram; ...
- The **history** h is the context; the model is a table $p(w \mid h)$.
- Sequences are padded with <bos> so the first symbol has a history.

Maximum likelihood (Appendix A.3): with $c(h, w)$ the count of h followed by w,

$$\hat{p}(w \mid h) \;=\; \frac{c(h, w)}{c(h)}, \qquad c(h) = \sum_{w} c(h, w).$$

Toy corpus

18 tokens, 9 types; the period is a token. Kept in Portuguese so that every number matches the notes:

```
o gato dormiu no sofá .  o cachorro dormiu no tapete .  o gato comeu
                                                    no tapete .
```

o the · gato cat · dormiu slept · no on the · sofá couch · cachorro dog · tapete rug · comeu ate

Unigram counts: o 3, no 3, . 3, gato 2, dormiu 2, tapete 2, sofá 1, cachorro 1, comeu 1.

In English, "the" would absorb both o and no and all counts would change.

Toy corpus: bigram MLE

h	w	$c(h, w)$	$c(h)$	$\hat{p}(w \mid h)$
dormiu	no	2	2	1.00
no	tapete	2	3	0.67
no	sofá	1	3	0.33
o	gato	2	3	0.67
o	cachorro	1	3	0.33
cachorro	comeu	0	1	**0.00**

Test sentence o cachorro comeu no sofá . ("the dog ate on the couch"): reasonable, but (cachorro, comeu) was never seen.
Probability zero. $\mathrm{PPL} = \infty$.

Sparsity

An n-gram model has $V^{n-1}(V-1)$ free parameters.

V	n	parameters
50,000	3	$\sim 10^{14}$
50,000	5	$\sim 10^{23}$

A corpus of 10^{11} tokens sees at most 10^{11} distinct trigrams: a fraction 10^{-3}, most of them once.

Curse of dimensionality in its crudest form. Either n is small, or the model is full of zeros and single-observation estimates.

Smoothing 1: additive (Laplace, add-α)

Add $\alpha > 0$ to every count:

$$\hat{p}_\alpha(w \mid h) \;=\; \frac{c(h, w) + \alpha}{c(h) + \alpha V}.$$

Toy corpus, $\alpha = 1$: $\hat{p}(\texttt{comeu} \mid \texttt{cachorro}) = \frac{1}{10}$, $\hat{p}(\texttt{dormiu} \mid \texttt{cachorro}) = \frac{2}{10}$. The zero is gone.

The problem. For most histories $c(h)$ is small and αV dominates.
`cachorro` (seen once) now gives 80% of its mass to never-seen successors.
With $V = 50{,}000$: over 99.9%.

It is MAP with a symmetric Dirichlet prior (Appendix A.4): "all V symbols are equally plausible after any history". False.

Smoothing 2: interpolation (Jelinek–Mercer)

Mix orders. The lower order is less sparse and serves as fallback:

$$\hat{p}_{\text{int}}(w \mid h_{n-1}) = \lambda_n \, \hat{p}^{\text{MLE}}(w \mid h_{n-1}) + (1 - \lambda_n) \, \hat{p}_{\text{int}}(w \mid h_{n-2}),$$

down to the unigram.

- λ_n is tuned on *validation* — on training the optimum is $\lambda = 1$.
- Better variants let λ depend on $c(h)$: more evidence, more trust in the higher order.
- **This is what Lab 1 asks for.**

Interpolation: the toy sentence

Bigram + unigram, $\lambda = 0.7$: $\hat{p}_{\text{int}} = 0.7\,\hat{p}_2^{\text{MLE}} + 0.3\,\hat{p}_1$.

h	w	$\hat{p}_2^{\text{MLE}}$	$\hat{p}_1(w)$	$\hat{p}_{\text{int}}$
o	cachorro	0.333	0.056	$0.7 \cdot 0.333 + 0.3 \cdot 0.056 = 0.250$
cachorro	comeu	0	0.056	$0 + 0.3 \cdot 0.056 = 0.017$
comeu	no	1	0.167	$0.7 + 0.3 \cdot 0.167 = 0.750$
no	sofá	0.333	0.056	0.250
sofá	.	1	0.167	0.750

Product $\approx 5.9 \cdot 10^{-4}$; $\mathrm{PPL} = (5.9 \cdot 10^{-4})^{-1/5} \approx 4.4$. Finite, and reasonable for 9 types.

Smoothing 3: Kneser–Ney — absolute discounting

Subtract a constant $D \in (0, 1]$ from each positive count; give the freed mass to the lower order:

$$\hat{p}_{\mathrm{KN}}(w \mid h) \;=\; \frac{\max\big(c(h, w) - D,\, 0\big)}{c(h)} \;+\; \frac{D \cdot N_{1+}(h\,\bullet)}{c(h)}\, p_{\mathrm{cont}}(w),$$

$N_{1+}(h\,\bullet)$ = number of distinct symbols that follow h.

Why subtract, not add? Counts of rare events are overestimates; the empirical correction is roughly a constant subtraction.

Kneser–Ney — continuation probability

The lower order is *not* the unigram $c(w)/N$ but

$$p_{\text{cont}}(w) \;=\; \frac{\left|\{h : c(h,w) > 0\}\right|}{\left|\{(h,w') : c(h,w') > 0\}\right|} :$$

in how many distinct contexts w appears — not how many times.

"San Francisco". "Francisco" is frequent, but almost only after "San".
The lower order is consulted only when the context is *new* — and after a new context, "Francisco" is a terrible bet. p_{cont} knows: few distinct contexts.

Toy corpus (12 bigram types): o has $\hat{p}_1 = 3/18$ but $p_{\text{cont}} = 1/12$ — frequent, yet predictable from context.

Kneser–Ney — status

- Modified interpolated KN (Chen & Goodman, 1998): three discounts for counts 1, 2, ≥ 3. State of the art for about fifteen years.
- The final estimator is **no longer maximum likelihood** — a heuristic, justified empirically.
- A posteriori, a Bayesian reading: Teh (2006), hierarchical Pitman–Yor.
- p_{cont} is the *unique* lower order consistent with the unigram counts — Appendix A.5.

What n-grams cannot do, whatever the data

1. No generalization across similar contexts.
`gato dormiu no` and `cachorro dormiu no` share no statistics. Tokens are integer indices: `gato` is as far from `cachorro` as from `tapete`.

2. Fixed window.
Anything more than n positions away is invisible: agreement, pronouns, the previous paragraph. Increasing n worsens sparsity exponentially.

Why neural networks: Bengio et al. (2003)

Each token w gets a dense vector $e_w \in \mathbb{R}^d$, $d \ll V$ — the **embedding**. Matrix $E \in \mathbb{R}^{V \times d}$, learned.

$$p(w \mid h) = \mathrm{softmax}\big(W_2 \tanh(W_1 [e_{h_1}; \ldots; e_{h_{n-1}}] + b_1) + b_2\big)_w.$$

Where the gain comes from. If `gato` and `cachorro` appear in similar contexts, gradient descent pushes their vectors together — one computation serves both, the loss drops. "cat slept on the" now transfers to "dog slept on the".

Counting versus optimization

The neural network buys generalization at the price of replacing counting with optimization.

- Estimation is no longer count-and-divide: it is minimizing a non-convex function of millions of parameters — initialization, learning rate, regularization, instability.
- Defect 2 (fixed window) is solved by attention (Week 2): condition on the whole prefix, with no n fixed in advance.

Do not underestimate the baseline

- Suffix arrays over the whole corpus allow n-gram queries of *unbounded* order on trillions of tokens.
- Liu et al. (2024, *Infini-gram*): interpolating with a large neural LM *still* lowers its perplexity — quotations, names, memorizable sequences.

The n-gram of Lab 1 is the baseline the Transformer of Lab 3 has to beat, in the same units.

Outline

1. The problem
2. How to measure
3. *n*-grams
4. Tokenization
5. Bridge
6. Appendix: proofs and code

What the tokenizer does

Text (Unicode characters; on disk, bytes) $\longrightarrow$ integers in $\{1, \ldots, V\}$, and back.
Lossless, at least for "normal" text.

- Trained *before* the model, on a possibly different corpus. Then frozen.
- Defines $\mathcal{V}$ — hence the size of the embedding matrix $E \in \mathbb{R}^{V \times d}$ and of the output layer, the two largest layers of a small model.

Why subwords

Unit	V	Problem
Byte	256	sequences 4–5$\times$ longer; attention $O(T^2)$; little information per step
Character	10^2–10^5	open vocabulary in Unicode; same length problem
Word	$\gg 10^5$	out-of-vocabulary words; morphology ignored; huge E; languages without spaces
Subword	$3 \cdot 10^4$–$2.5 \cdot 10^5$	**the compromise**: closed vocabulary, reasonable length, rare words built from frequent pieces

The vocabulary size is a trade-off

- **Larger V**: shorter sequences — more text per context window, cheaper attention. But E grows, and each token is seen fewer times: rare tokens are poorly trained.
- **Smaller V**: the opposite.

Llama 2	GPT-4	Llama 3	Qwen	Gemma
32k	∼100k	128k	∼152k	256k

Trend: upward, driven by multilinguality.

Byte-Pair Encoding: training

Gage (1994), a *compression* algorithm; Sennrich et al. (2016), adapted to translation.

1. **Pre-tokenize**: split into "words". Merges never cross a word boundary.
2. Start from the **base alphabet**: characters, or the 256 bytes.
3. Count adjacent symbol pairs, weighted by word frequency.
4. **Merge** the most frequent pair $(a, b) \to ab$; record the rule.
5. Repeat until $|\mathcal{V}|$ reaches the target.

GPT-2: $\sim$50k merges over 256 bytes.

Byte-Pair Encoding: encoding new text

Decompose each word into the base alphabet; apply the merges **in the order they were learned**.

Same vocabulary, different merge order $\Rightarrow$ different segmentation.
The vocabulary does not define the tokenizer; the ordered list of merges does.

Worked example: the corpus

Four Portuguese words sharing the stem `aix`:

$$\texttt{baixo} \times 5, \quad \texttt{baixa} \times 3, \quad \texttt{abaixo} \times 2, \quad \texttt{caixa} \times 4.$$

Base alphabet: {`a`, `b`, `c`, `i`, `o`, `x`}.

Count the adjacent pairs (weighted by frequency):

$$(\texttt{a}, \texttt{i})\ 14, \qquad (\texttt{i}, \texttt{x})\ 14, \qquad (\texttt{b}, \texttt{a})\ 10, \qquad (\texttt{x}, \texttt{o})\ 7, \qquad \ldots$$

A **tie** already: (a,i) and (i,x). The tie-breaking rule is part of the tokenizer.

Worked example: the five merges

#	Top pair	Merge	Corpus after the merge
1	(a,i) 14	ai	b·ai·x·o, b·ai·x·a, a·b·ai·x·o, c·ai·x·a
2	(ai,x) 14	aix	b·aix·o, b·aix·a, a·b·aix·o, c·aix·a
3	(b,aix) 10	baix	baix·o, baix·a, a·baix·o, c·aix·a
4	(baix,o) 7	baixo	baixo, baix·a, a·baixo, c·aix·a
5	(c,aix) 4	caix	baixo, baix·a, a·baixo, caix·a

Final vocabulary, 11 symbols:
{a, b, c, i, o, x, ai, aix, baix, baixo, caix}.

On the board: cover row 4 and ask for the prediction before counting.

Worked example: what to notice

- **Morphemes emerge.** In three merges, `aix` — shared by all four words. Nothing linguistic in the algorithm; morphemes are just the sequences that repeat.
- **Intermediate symbols stay.** `ai` no longer appears in the training corpus, but it is in the vocabulary and may appear in new text.

A word never seen: `abaixa`.

$$\texttt{a}\cdot\texttt{b}\cdot\texttt{a}\cdot\texttt{i}\cdot\texttt{x}\cdot\texttt{a} \xrightarrow{1} \texttt{a}\cdot\texttt{b}\cdot\texttt{ai}\cdot\texttt{x}\cdot\texttt{a} \xrightarrow{2} \texttt{a}\cdot\texttt{b}\cdot\texttt{aix}\cdot\texttt{a} \xrightarrow{3} \texttt{a}\cdot\texttt{baix}\cdot\texttt{a}$$

Merges 4 and 5 do not apply. Three tokens, built from known pieces.

Minimal BPE: training

```
from collections import Counter

def train_bpe(words, n_merges):
    corpus = Counter(tuple(w) for w in words)       # word -> tuple of symbols
    merges = []
    for _ in range(n_merges):
        pairs = Counter()                                   # 1. count adjacent pairs
        for symbols, freq in corpus.items():
            for a, b in zip(symbols, symbols[1:]):
                pairs[(a, b)] += freq
        if not pairs:
            break
        (a, b), _ = pairs.most_common(1)[0]             # 2. most frequent pair
        merges.append((a, b))
        new = {}
        for symbols, freq in corpus.items():            # 3. replace (a, b) by a+b
            out, i = [], 0
            while i < len(symbols):
                if symbols[i:i + 2] == (a, b):
                    out.append(a + b); i += 2
                else:
                    out.append(symbols[i]); i += 1
            new[tuple(out)] = freq
        corpus = new
    return merges                                           # the order IS the tokenizer
```

Minimal BPE: encoding

```
def encode(word, merges):
    symbols = list(word)
    for a, b in merges:                  # in the learned order
        out, i = [], 0
        while i < len(symbols):
            if symbols[i:i + 2] == [a, b]:
                out.append(a + b); i += 2
            else:
                out.append(symbols[i]); i += 1
        symbols = out
    return symbols

c = {"baixo": 5, "baixa": 3, "abaixo": 2, "caixa": 4}
words = [w for w, n in c.items() for _ in range(n)]
merges = train_bpe(words, 5)          # [('a','i'), ('ai','x'), ('b','aix'), ...]
print(encode("abaixa", merges))       # ['a', 'baix', 'a']
```

Byte-level version with pre-tokenization: Lab 1.

Byte-level BPE (GPT-2 onward)

Base alphabet = the **256 byte values**; text is converted to UTF-8 first.

1. **No unknown token, ever.** Any language, emoji, binary garbage: a byte sequence. Worst case, byte by byte.
2. **No text has probability zero** at the tokenizer level — Pitfall 3 closed from the preprocessing side.
3. A multi-byte character can be split across tokens; a decoded prefix may not be valid UTF-8. Visible in *streaming*.

Bytes are renamed to printable characters for storage — Appendix A.6.

Regex pre-tokenization

Before BPE, the text is cut by a regex; merges never cross the cuts. GPT-2:

```
's|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+
```

Contractions; optional space + letters; optional space + digits; optional space + punctuation; whitespace.

- **The space belongs to the following word.** `" house"` and `"house"` are different tokens, with different embeddings.
- **Letters and digits never mix**; punctuation stays separate.
- GPT-4 / Llama 3 add a **three-digit limit**: `\p{N}{1,3}`. Why: next section.

WordPiece (BERT)

Same merges as BPE, different criterion:

$$\text{score}(a, b) \;=\; \frac{c(ab)}{c(a)\,c(b)},$$

the pointwise mutual information: how much more often the pair occurs *than expected* under independence.

- Favors pairs whose parts are rare on their own; equivalent to maximizing unigram likelihood.
- Continuations carry `##`: `develop`, `##ment`.
- Encoding is **greedy longest prefix**, not re-applied merges.

Unigram (Kudo, 2018) and SentencePiece

Unigram: start from a large candidate vocabulary and *prune*.

- A segmentation $s = (s_1, \ldots, s_k)$ has probability $\prod_i p(s_i)$.
- Estimate p by EM (segmentation is latent); remove the tokens whose loss is smallest; repeat.
- Encode by Viterbi; *sample* segmentations in training — subword regularization.

SentencePiece: a library (BPE or Unigram) on raw text, no whitespace pre-tokenization; the space is the symbol _. T5, Llama 1–2, Gemma. Llama 3 moved to byte-level BPE (`tiktoken`).

Consequences of tokenization

Each of the following is a behavior of **production models** whose cause is the tokenizer — not the architecture, not the training.

1. Arithmetic
2. Languages other than English
3. Code
4. Spelling, counting letters, rhyming
5. Token boundaries and the trailing space
6. Under-trained tokens
7. The compression lens

1. Arithmetic

Suppose 1234 is one token (a frequent year) but 12345 is 123 + 45.

- The model sees numbers in **inconsistent units**: one digit here, four there.
- Addition needs digit alignment by position — impossible to learn in general.

Responses: the three-digit limit (GPT-4, Llama 3); digit-by-digit tokenization (Llama 1–2); grouping from the *right*, aligned with decimal places.

2. Languages other than English

Vocabularies learned on English corpora have higher **fertility** — tokens per word — elsewhere. GPT-2: *development* is one token; *desenvolvimento* is four or five.

- More tokens per content: more money, more latency.
- Less content per context window.
- Fewer "examples" per unit of text in pretraining.
- Per-token perplexity not comparable across languages.

Petrov et al. (2023): up to 3–5× English fertility for some languages.
Lab 1: measure it in Portuguese and English.

3. Code 4. Letters

Code. GPT-2's regex isolates indentation spaces one by one: four spaces = four tokens before any content. Code models added tokens for runs of spaces and tabs.
General point: what the tokenizer's corpus lacked becomes expensive.

Spelling, counting letters, rhyming. The model never sees characters, only indices. "How many *r*'s in *strawberry*?" requires reconstructing a token's spelling from statistics.
The information was destroyed before reaching the model.

5. Token boundaries 6. Under-trained tokens

The trailing space. `" house"` $\neq$ `"house"`. A prompt ending in a space is out of distribution. Inference libraries patch it with *token healing*.

Under-trained tokens. Tokenizer and model are trained on different corpora. A token frequent in one, absent from the other, keeps a random embedding — and produces erratic output when it shows up (Land & Bartolo, 2024: `SolidGoldMagikarp`).

7. The compression lens

- BPE *is* dictionary compression — it was invented as such.
- The vocabulary is a variable-length code: frequent sequences get short codes, rare ones are spelled out.
- The LM compresses what is left after this first stage.

Removing the discrete stage: ByT5 (bytes, long sequences); Byte Latent Transformer (Pagnoni et al., 2024) — adaptive byte patches sized by predicted entropy.
In 2026: research, not production.

Outline

1. The problem
2. How to measure
3. *n*-grams
4. Tokenization
5. Bridge
6. Appendix: proofs and code

Lab 1 (M1P2), this week

1. **Byte-level BPE** from scratch on a small Portuguese corpus.
2. n-**grams**, $n = 1, 2, 3$, with interpolation; λ on validation.
3. **Evaluation**: perplexity *and* bits per byte, on a held-out test set.
4. **Fertility** of your tokenizer vs. GPT-2's, Portuguese and English.

Item 3 is the baseline the 10M Transformer of Lab 3 must beat — same units, same test set.

Week 2 starts from $E \in \mathbb{R}^{V \times d}$ — whose size was decided today — and moves to attention.

Exercises (due with Lab 1)

1. **Perplexity is not additive.** Show that on a concatenation it is the weighted geometric mean $\exp\left(\frac{T_1 \ln P_1 + T_2 \ln P_2}{T_1 + T_2}\right)$.
2. **Bits per byte.** A: $V = 32\text{k}$, $\mathrm{PPL} = 12$. B: $V = 128\text{k}$, $\mathrm{PPL} = 15$, 20% fewer tokens. Which compresses better? What is missing for absolute values?
3. **BPE by hand.** The five merges; encode `baixinho`; why can GPT-2's regex never merge o with a following space?
4. **MLE and MAP.** Prove $c(h, w)/c(h)$ is the MLE; show add-α is MAP under $\mathrm{Dir}(\alpha + 1)$; why is that prior bad for large V?
5. **gzip as an LM.** $p_C \propto 2^{-|C(x)|}$ with `zlib`; compare with character bigrams.
6. **Discussion.** The loss never reaches zero. How would you estimate H for a corpus, and with what limitations?

Outline

1. The problem
2. How to measure
3. *n*-grams
4. Tokenization
5. Bridge
6. Appendix: proofs and code

A.1 Kraft, forward direction

Claim. Prefix code with lengths $\ell(x) \Rightarrow \sum_x 2^{-\ell(x)} \leq 1$.

To the codeword $c = c_1 \cdots c_\ell$ associate the binary interval

$$I(c) = \left[0.c_1 \cdots c_\ell, \;\; 0.c_1 \cdots c_\ell + 2^{-\ell} \right) \subset [0, 1).$$

- c prefix of $c' \Rightarrow I(c') \subset I(c)$.
- Prefix code $\Rightarrow$ the intervals are pairwise disjoint.
- All inside $[0, 1) \Rightarrow$ total length $\sum_x 2^{-\ell(x)} \leq 1$. $\square$

Converse: sort the lengths and stack intervals of length $2^{-\ell_i}$ from the left; each starts at a multiple of $2^{-\ell_i}$, i.e. is a word of ℓ_i bits.

A.1 Shannon's bound and the Shannon code

Claim. $\sum_x q(x)\ell(x) \geq H_2(q)$ for any prefix code.
Let $r(x) = 2^{-\ell(x)}/Z$, $Z = \sum_x 2^{-\ell(x)} \leq 1$. Then

$$\sum_x q(x)\ell(x) - H_2(q) = \sum_x q(x) \log_2 \frac{q(x)}{2^{-\ell(x)}} = \mathrm{KL}(q \,\|\, r) - \log_2 Z \;\geq\; 0. \;\square$$

Shannon code. $\ell(x) = \lceil -\log_2 q(x) \rceil$ satisfies Kraft and

$$\sum_x q(x)\ell(x) < H_2(q) + 1.$$

Within one bit. Arithmetic coding removes that bit *per symbol*.

A.2 AEP: the i.i.d. case of SMB

Claim. X_t i.i.d. $\sim q \Rightarrow -\frac{1}{T}\log q(X_{1:T}) \to H(q)$ almost surely.

- Independence: $\log q(X_{1:T}) = \sum_t \log q(X_t)$.
- $Y_t = -\log q(X_t)$ are i.i.d. with $\mathbb{E}[Y_t] = H(q) < \infty$.
- Strong law of large numbers. $\square$

With $-\log p_\theta(X_t)$ instead: the limit is $H(q) + \mathrm{KL}(q \,\|\, p_\theta)$.
Ergodic Markov chains: same argument with the ergodic theorem.
General stationary ergodic processes: SMB proper (Cover & Thomas 16.8).

A.3 MLE for n-grams

Log-likelihood grouped by (history, symbol):

$$\ell(p) = \sum_h \sum_w c(h, w) \log p(w \mid h), \qquad \sum_w p(w \mid h) = 1 \text{ for each } h.$$

One problem per history. Lagrangian: $\sum_w c(h, w) \log p(w \mid h) - \mu(\sum_w p(w \mid h) - 1)$.

Differentiate: $p(w \mid h) = c(h, w)/\mu$. Sum over w: $\mu = c(h)$. Concave objective, linear constraint: global maximum. $\square$

For $c(h, w) = 0$ the objective ignores $p(w \mid h)$; any mass there is stolen from the others. **Maximum likelihood wants zeros on everything unseen.**

A.4 Additive smoothing is MAP

Prior $\mathrm{Dir}(\beta, \ldots, \beta)$ on $p(\cdot \mid h)$, $\beta > 1$. Posterior $\propto \prod_w p(w \mid h)^{c(h,w)+\beta-1}$ — the MLE problem with $c(h, w) \to c(h, w) + \beta - 1$:

$$\hat{p}_{\mathrm{MAP}}(w \mid h) = \frac{c(h, w) + \beta - 1}{c(h) + V(\beta - 1)} \qquad (\text{add-}\alpha,\ \alpha = \beta - 1). \square$$

- The posterior *mean* gives $\alpha = \beta$; both conventions exist.
- **The point:** the prior is symmetric — every symbol equally plausible after every history. False for large V.
- Kneser–Ney $\approx$ a prior informed by the lower order (Teh, 2006).

A.5 Kneser–Ney continuation: setup

Interpolated absolute discounting, bigram case, $0 < D \leq 1$:

$$\hat{p}(w \mid v) = \frac{\max(c(v, w) - D,\, 0)}{c(v)} + \frac{D\, N_{1+}(v\,\bullet)}{c(v)}\, p_{\text{low}}(w),$$

with p_{low} unknown.

Marginal constraint: regenerating the corpus from the observed histories must reproduce each unigram count,

$$\sum_{v} c(v)\, \hat{p}(w \mid v) \;=\; c(\bullet\, w) \quad \text{for every } w.$$

A.5 Kneser–Ney continuation: solution

Substitute. Since $D \le 1$ and counts are integers,

$$\sum_v \max(c(v,w) - D, 0) = c(\bullet\, w) - D\, N_{1+}(\bullet\, w), \qquad \sum_v N_{1+}(v\, \bullet) = N_{1+}(\bullet\, \bullet).$$

The constraint becomes

$$c(\bullet\, w) - D\, N_{1+}(\bullet\, w) + D\, p_{\text{low}}(w)\, N_{1+}(\bullet\, \bullet) = c(\bullet\, w),$$

so

$$\boxed{p_{\text{low}}(w) = \frac{N_{1+}(\bullet\, w)}{N_{1+}(\bullet\, \bullet)}}$$ the continuation probability. $\square$

Checked on the toy corpus with $D = 0.75$: every $\hat{p}(\cdot \mid v)$ sums to 1 and the marginals match exactly.

A.5 Modified Kneser–Ney (Chen & Goodman, 1998)

Three discounts D_1, D_2, D_{3+} for counts $1, 2, \geq 3$, estimated from the corpus with $n_k =$ number of n-grams seen exactly k times:

$$Y = \frac{n_1}{n_1 + 2n_2}, \qquad D_1 = 1 - 2Y\frac{n_2}{n_1}, \qquad D_2 = 2 - 3Y\frac{n_3}{n_2}, \qquad D_{3+} = 3 - 4Y\frac{n_4}{n_3}.$$

- A leave-one-out argument: how much would the count drop if one occurrence were removed?
- Lower orders remain continuation distributions, recursively.
- On tiny corpora (toy: $n_3 = 0$) the formulas degenerate; thousands of tokens suffice in practice.

A.6 GPT-2's byte→Unicode mapping

The vocabulary is saved as JSON strings, but many bytes are not printable and the space breaks serialization.

Fixed bijection between the 256 bytes and 256 printable characters:

- 188 bytes already printable in Latin-1 map to themselves;
- the other 68 go, in order, to U+0100 onward (Ā, ā, Ă, ...).

Hence `" the"` shows up as `Ġthe`: Ġ (U+0120) is byte 32, the space. A renaming only; no effect on the model.

B.1 Interpolated n-grams: training

```
import math
from collections import defaultdict, Counter
BOS, EOS = "<bos>", "<eos>"

class InterpolatedNGram:
    def __init__(self, n, lambdas):              # lambdas: unigram -> order n, sum to 1
        assert len(lambdas) == n and abs(sum(lambdas) - 1) < 1e-9
        self.n, self.lambdas = n, lambdas
        self.counts = [defaultdict(Counter) for _ in range(n)]   # counts[k][h][w]
        self.vocab = set()

    def train(self, sequences):
        for seq in sequences:
            toks = [BOS] * (self.n - 1) + list(seq) + [EOS]
            self.vocab.update(toks)
            for t in range(self.n - 1, len(toks)):
                for k in range(self.n):              # for each order k+1
                    h = tuple(toks[t - k:t])         # history of length k
                    self.counts[k][h][toks[t]] += 1
```

B.1 Interpolated n-grams: probability and evaluation

```
    def prob(self, w, history):
        p = 0.0
        for k in range(self.n):
            h = tuple(history[len(history) - k:]) if k > 0 else tuple()
            c_h = self.counts[k].get(h)
            p_mle = c_h[w] / sum(c_h.values()) if c_h else 0.0   # unseen history: 0
            p += self.lambdas[k] * p_mle
        return p

    def evaluate(self, sequences, n_bytes=None):
        log_probs = []
        for seq in sequences:
            toks = [BOS] * (self.n - 1) + list(seq) + [EOS]
            for t in range(self.n - 1, len(toks)):
                p = self.prob(toks[t], toks[:t])
                log_probs.append(math.log(p) if p > 0 else float("-inf"))
        ce = -sum(log_probs) / len(log_probs)
        bpb = (-sum(log_probs) / math.log(2)) / n_bytes if n_bytes else None
        return ce, math.exp(ce), bpb
```

Toy corpus, $n = 2$, $\lambda = (0.3, 0.7)$: $\mathrm{PPL} = 4.49$ vs. 4.43 by hand — here the unigram counts <bos>/<eos>. Tune λ on validation.

B.2 Fertility of a tokenizer

```
import re
import tiktoken                          # pip install tiktoken

def fertility(text, encode):             # tokens per word
    words = re.findall(r"\w+", text)
    return len(encode(text)) / len(words)

enc_gpt2 = tiktoken.get_encoding("gpt2")
pt = "O desenvolvimento de modelos exige corpora amplos."
en = "The development of models needs large corpora."
for name, txt in [("pt", pt), ("en", en)]:
    print(name, round(fertility(txt, enc_gpt2.encode), 2))
```

Expected: much higher fertility in Portuguese. In the lab, use long texts and your own BPE for comparison.

References: textbooks and classic papers

- Jurafsky & Martin, *Speech and Language Processing*, 3rd ed., chs. 2–3.
- Raschka, *Build a Large Language Model (From Scratch)*, Manning, 2024, ch. 2.
- Cover & Thomas, *Elements of Information Theory*: chs. 4, 5; §13.3; §16.8.
- Shannon, Prediction and entropy of printed English, *BSTJ*, 1951.
- Bengio et al., A neural probabilistic language model, *JMLR*, 2003.
- Chen & Goodman, An empirical study of smoothing techniques, Harvard TR, 1998.
- Teh, A hierarchical Bayesian LM based on Pitman–Yor processes, *ACL*, 2006.
- Gage, A new algorithm for data compression, 1994. Sennrich, Haddow & Birch, subword units, *ACL*, 2016.
- Schuster & Nakajima, WordPiece, *ICASSP*, 2012. Kudo, Subword regularization, *ACL*, 2018. Kudo & Richardson, SentencePiece, *EMNLP*, 2018.
- Radford et al., GPT-2, 2019.

References: recent work, code

- Delétang et al., Language modeling is compression, *ICLR*, 2024.
- Petrov et al., Tokenizers introduce unfairness between languages, *NeurIPS*, 2023.
- Land & Bartolo, Fishing for Magikarp, *EMNLP*, 2024.
- Liu et al., Infini-gram, *COLM*, 2024.
- Pagnoni et al., Byte Latent Transformer, 2024.

Code and video

- Karpathy, *Let's build the GPT tokenizer* and `minbpe` — reference for Lab 1.
- `tiktoken` (OpenAI), `tokenizers` (Hugging Face).